

Exact Recovery of PDN Grid Specifications from Layout Geometry: A Reverse-Engineering Study on OpenROAD’s `pdngen`

Feilian Huang
Independent Researcher
fffeilian@gmail.com

September 26, 2026

Abstract

Power delivery network (PDN) grid specifications—rail widths, stripe pitches and offsets, layer connectivity, net phasing—are proprietary design inputs. We prove an analytic identifiability theorem for regular grid generators and validate it on OpenROAD’s `pdngen`: an inference algorithm recovers every geometry-determining parameter exactly, with all degenerate regimes characterized explicitly—the dropout-induced naming duality, phasing coincidence, single-stripe layers, zero-yield connect pairs, dbu rounding, float-truncation near-duals, and the $\mathbb{Z}_2$ naming symmetry. On four designs across three PDKs recovery is exact with wire-for-wire round-trip regeneration; a label-free variant closes the gap on real GDSII polygons after a constructive name-strip. Randomized sweeps of 1,600 configurations with archived truth—1600/1600 inference and round-trip, 1595/1600 exact, zero failures—certify implementation correctness against the forward model. The recovered spec regenerates a legal PDN on a modified floorplan where naive geometry copy fails. Decoys buy only intent attribution; jitter coarsens recovery; via dropout is falsified. Exact recovery holds on current C++ OpenROAD (N=200, all exact); legacy-Tcl results are archival.

1 Introduction and Threat Model

Hardware reverse engineering (RE) classically proceeds from silicon upward: delayering, imaging, circuit extraction, and netlist reconstruction [4, 6, 5, 9]. Most RE research targets logic functionality, while the power delivery network is treated as background infrastructure. We show that the PDN deserves attention in its own right: the *rules* that generate it can be recovered exactly from the extracted layout database.

Threat model. The adversary obtains the physical layout database of a placed-and-routed design (DEF/GDSII/OASIS, e.g. from an untrusted foundry or a leaked tapeout database). Concretely, our extractor consumes the post-extraction ODB view: per-shape records carry the power/ground net identity (from DEF SPECIALNETS USE POWER/GROUND), the shape kind (FOLLOWPIN vs. STRIPE), and via bottom/top layer spans, plus die/core/row metadata. No access to the original PDN configuration scripts, PDK-private design rules beyond the public LEF, or the EDA tool’s internal state is assumed. The adversary’s goal is to recover the PDN *generation* specification—the parameter bundle sufficient to reconstruct the grid geometry (whether or not that bundle is accepted verbatim by the generator’s own configuration language)—rather than merely tracing wires. We evaluate two input regimes: the labeled layout-database view above, and de-labeled polygons, for which we add connectivity tracing and power/ground net identification before inference runs (§6.5)—the

de-labeled regime validated on a real GDSII stream-out, the actual untrusted-foundry input. On labels: the shape-kind labels in the labeled view are written by the generator itself, so that view leaks part of the answer; §6.5 shows this step is recoverable from geometry (a rule classifier with 0 misclassifications)—which is what makes the de-labeled variant meaningful rather than a formality. What we do *not* close is the imaging gap: per-layer polygon attribution, cut-layer grouping, and stack order are assumed known, and layering noise (misalignment, missed/false cuts) is unmodeled—true imaged polygons remain future work.

The spec matters even though the geometry can always be cloned, for two reasons. First, *regeneration under modification*: re-spinning a block or porting the layout to a derivative floorplan needs the generation rules, not a frozen snapshot of shapes—§2 shows the recovered spec replaying legally on a modified floorplan where naive polygon copy shorts. The spec is the portable, editable form of the design intent; the geometry is one instantiation of it. We are candid about the IP value: most spec values are process-derived rather than deep design secrets; what recovery buys is the ability to reconstruct the grid exactly and replay it—a compact, editable representation that polygon copying cannot provide once the floorplan changes. Second, and more narrowly, *partial observation*: an attacker who recovers the rules from a sampled die region can predict the PDN across the full chip, including areas never imaged—something static geometry cloning cannot do. We quantify the limits of this in §6.4: on sky130hd/gcd, a 20%-area strip crop or a 10%-area square crop recovers every parameter exactly and regenerates the full die, while a thin 10% strip fails for lack of met4 samples. Recovery from a region therefore depends on the crop shape and on each (layer, net) presenting enough period instances; and we stress these are labeled-database crops—the harder label-free imaged-polygon setting remains future work.

We instantiate this attack against OpenROAD’s `pdngen`, the power-grid generator of the open-source OpenROAD flow [1, 2]. We chose `pdngen` because its source is public, which lets us verify our forward model line-by-line, and because its rule language (`pdngen::specify_grid`) is representative of industrial PDN specification styles (rails, stripes, rings, via connectivity [3]).

Contributions.

1. A verified forward model of `pdngen` grid generation, including the stripe-offset reference frame, the boundary stripe-dropout rule, and the two VSS phasing mechanisms; plus a deterministic inference algorithm recovering the minimal geometry-determining grid specification from extracted PG geometry.
2. An analytic identifiability theorem for the parameter-to-geometry map on the live-parameter quotient space, plus randomized sweeps of 1,600 configurations with archived truth, reaching 1600/1600 inference and round-trip, 1595/1600 exact (99.69%), zero failures, and 5 genuine geometric duals—certifying that the implementation realizes the forward model; degenerate regimes explicitly characterized (phasing coincidence, single-stripe layers, zero-yield connect pairs, dbu rounding, truncation near-duals, and a dropout-induced naming duality with an analytic existence condition); a proposed (start, offset) duality rejected analytically.
3. A label-free attack variant: from de-labeled ODB polygons, connectivity tracing and PDK-anchored net identification recover the full specification byte-identical to the labeled pipeline, up to a $\mathbb{Z}_2$ VDD/VSS naming symmetry — validated on a real GDSII stream-out, the untrusted-foundry input; plus an end-to-end demo showing the recovered spec regenerates a legal PDN on a modified floorplan where naive geometry copy fails.
4. Recovery extended to core rings (6/6 parameters exact) and macro grids (straps and connect chain exact), each with exact round-trip regeneration; and a parameter-semantics transfer

analysis to Innovus `addStripe` (document level—stripe geometry transfers, via/connect inference does not).

5. An evaluation of anti-RE PDN obfuscations (interleaved decoy grids, pitch jitter, via dropout) with attacker/defender-side measurements, including falsified defenses, negative controls, and explicitly qualitative IR comparisons.

2 End-to-end motivation: the spec as executable intent

The spec’s value over frozen geometry is *regeneration under modification*, which we demonstrate end-to-end before turning to the machinery. Starting from the recovered configuration for the sky130hd gcd design—`pdn_inferred.cfg`, holding rails met1 w0.480; straps met4 w1.600, p27.140, o13.570 and met5 w1.600, p27.200, o13.600; connect pairs met1–met4 and met4–met5; stripes start with POWER (recovered by the label-free pipeline of §6.5)—we modified the floorplan by inserting a $50 \times 50 \mu\text{m}$ fake macro (`FAKE_MACRO_0`) at the core center and replayed `pdngen` with the recovered `stdcell` spec reused verbatim, plus only the macro-grid declaration a designer would standardly write for the new floorplan (`orient, blockages "met4 met5", halo`)—which carries no stripe geometry. The run completes with 0 errors and no PDN-0042 or PDN-0079 warnings, producing a legal, self-avoiding PDN: 141 wires / 2,785 vias; 8 stripes intersecting the macro are each cut into two segments whose 16 ends stop exactly at the halo boundary; all 163 surviving met4×met5 crossings carry their via4 stack (0 missing); 0 wires and 0 vias enter the macro rectangle. The control—naively copying the original 3,040 polygons onto the new floorplan (die and core unchanged, the most favorable case for copying)—leaves 8 stripes shorting straight through the macro and 122 via rectangles landing inside it (46 distinct stack positions, 8 of them dead met4–met5 stacks). No rigid transform can legalize all 3,040 shapes at once, and hand-repairing cut stripes and dead vias does not scale with design size; parametric regeneration is the only systematic route. The thesis in one line: *the spec is executable intent; the geometry is a one-time instance*. This is replay inside the same generator, not cross-tool migration; the macro-grid declaration is designer-standard rather than inferred (the original design has no macro, so no macro-rule information exists in its geometry); and met1 rails still pass under the macro, since `blockages` covers stripe layers only, consistent with the platform’s macro-grid semantics.

Why automation: a manual-recovery cost model. A natural objection is that recovering a handful of numbers from a regular grid is an exercise an engineer could do by hand from a layout viewer. For a single grid this is conceded: stripe fitting is a few least-squares measurements, and via-span assignment is pattern-based—identify the stack type once from a representative sample (e.g. via2/via3 touch no wire, so spans come from cut-layer tables plus stack alignment), then propagate—so an expert plausibly recovers one grid in hours. What automation buys is the campaign: seconds per configuration instead of hours, scaling to hundreds of configurations, which is what makes recovery-rate statistics, the duality catalog, and the falsified-defense verdicts possible at all. Recovering one grid by hand is possible; characterizing identifiability by hand is not.

3 Forward Model of `pdngen`

All claims in this section were verified against `PdnGen.tcl` at OpenROAD commit `f12e2f47` (March 2022 build). We describe the `stdcell` grid; macro grids follow the same stripe machinery.

Rails. One rail is generated per standard-cell row boundary, with the configured width, spanning the row width. Net assignment (VDD/VSS) is determined by row orientation (N rows: VDD on top; FS rows: flipped—observed on our sky130hd test designs; this follows the PDK’s standard-cell definitions). Notably, the configuration parameter `rails_start_with` appears zero times in `PdnGen.tcl`—it is never read and is therefore a dead parameter. Rail `pitch/offset`, when present, do not affect rail placement either (a missing pitch defaults to twice the row height in `check_rails`).

Straps. Let the stdcell area be the union of standard-cell rows. `pdngen` works on an expanded area (`stdcell_plus_area`): identical in x , but extended in y by half the maximum rail width on each side (the source comment reads “larger than the stdcell area by half a rail”). Stripe centers are computed relative to the reference point

$$ref = (stdcell_xMin, stdcell_yMin - max_rail_width/2). \quad (1)$$

The base grid (the net named by `stripes_start_with`, $POWER \rightarrow VDD$) is placed at $ref + offset + k \cdot pitch$ for integer $k \geq 0$ while the center is below $area_max - width$. A candidate stripe whose body is not fully inside the core is *dropped* (leading-stripe dropout): `PdnGen.tcl` skips a stripe at center c when $c - width/2 < near - 1$ dbu, i.e. in practice the first stripe vanishes when $offset < width/2$ (confirmed by targeted Tcl bisection on both strap layers). The other net’s grid is the same grid shifted by

$$shift = \begin{cases} pitch/2, & \text{no spacing given,} \\ spacing + width, & \text{otherwise.} \end{cases} \quad (2)$$

Thus VDD and VSS each form a complete pitch grid; they are not a single interleaved grid. We verified the reference frame numerically, e.g. sky130hd/gcd: first VDD met4 stripe center $23690 = 10120 + 13570$ dbu, and first VDD met5 stripe center $24240 = (10880 - 240) + 13600$ dbu, where 240 is half the 480 dbu rail width.

Vias and connect. `connect` declares logical layer pairs, e.g. `{met1 met4} {met4 met5}`. `pdngen` realizes each pair as a via stack spanning all intermediate layers, placed at same-net wire crossings: a pair may span several routing layers—asap7’s `{M1 M5}` generates a continuous four-segment stack ($M1 \rightarrow M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$, 106 vias per segment). Whether insertion succeeds is decided not by layer “adjacency” but by the geometric overlap of the two endpoint grids at each crossing: orthogonal (H×V) crossings succeed, while a parallel narrow rail under a wide stripe, whose overlap cannot cover the stripe’s full width, is skipped entirely with PDN-0042 warnings—which is why sky130hd’s `{met1 met5}` (both horizontal) yields *zero* vias. We return to this in Section 7.

4 Inference Algorithm

Input: extracted PG geometry (per-shape records: net, layer, shape, kind $\in \{\text{wire}, \text{via}\}$, bounding box; via records carry the bottom/top layer span), plus die/core/row metadata and the DEF `SPECIALNETS` section (to read power/ground net names from `USE POWER/GROUND`). The ground-truth configuration is never consulted. Note this is the labeled layout-database view described in the threat model—net identity and shape kinds come from the ODB/DEF metadata, not from unlabeled polygons.

Rails. FOLLOWPIN wires are grouped by layer; width is the modal cross-dimension. Rail positions are row-determined and need no further parameters.

Straps. STRIPE wires are grouped by (net, layer). Per layer and net, sorted stripe centers are fit by least squares to $c_i = c_0 + i \cdot p$ (pitch p rounded to the unit grid; residual gate 2 dbu, which rejects genuinely multimodal inputs loudly rather than silently accepting them). The net with the smaller first center is the base grid, fixing `stripes_start_with` (POWER or GROUND). Offset is (first base center $- ref$) with ref from Eq. (1). The VSS shift is compared against $pitch/2$: equality (no `spacing` emitted) versus $spacing = shift - width$.

connect. Via (bottom, top) spans are chained bottom-to-top through shared layers. Within each chain, the lowest *endpoint* rail layer paired with the first strap endpoint above it yields the rail→strap pair; consecutive strap endpoints within the chain yield strap→strap pairs. Intermediate layers of a stack are not endpoints: in asap7 the chain $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4 \rightarrow M5 \rightarrow M6$ collapses to $\{\{M1\ M5\} \{M5\ M6\}\} \rightarrow M2$, though a rail layer, is a middle layer of the $M1 \rightarrow M5$ stack, so the inferred pair is $M1 \rightarrow M5$, matching ground truth.

The output is a minimal grid configuration for `pdngen` containing only geometry-determining parameters; dead parameters (`halo`, `rails_start_with`, rail pitch/offset, grid name) are omitted.

5 Experimental Setup

We built OpenROAD (f12e2f47, March 2022) and Yosys 0.69 (oss-cad-suite) via conda, with OpenROAD-flow-scripts at commit 96eb3de (April 2022). Yosys is used only for synthesis; the PDN geometry under study is independent of the Yosys version. Four designs were run through floorplan and PDN generation: `sky130hd/gcd`, `nangate45/gcd`, `sky130hd/aes`, and `asap7/gcd`. This ORFS vintage uses the legacy `pdngen::specify_grid` syntax (via `pdn.cfg` / `grid_strategy` files), which is our inference target. Geometry was extracted directly from the layout database with the Tcl ODB API (`getSWires/getWires` over POWER/GROUND nets), because this build’s `write_def` degrades stripe wires to zero-width points. None of the four designs contains a core ring or a macro in its baseline flow; rings and macro grids are evaluated separately in §6.6.

Reproducibility. The repository <https://github.com/EDGAbab/pgrev> (MIT license) holds all inference and extraction scripts (`pg_infer.py`, the randomized-sweep driver `randscan.py`), the ground-truth configurations, and the `PdnGen.tcl` source snapshot used for forward-model verification.

6 Results

6.1 Exact parameter recovery

Table 1 compares inferred versus ground-truth parameters. *Every* geometry-determining parameter matches exactly, down to the layout database’s unit grid (width, pitch, offset, spacing; connect pairs and `stripes_start_with` identically). The asap7 case is notable: its ground truth selects between two grid strategies by a conditional, and inference correctly identified the active branch, including the `spacing`-based (rather than `pitch/2`) VSS phasing on both strap layers (M5 shift 192 dbu = 72 + 120; M6 shift 384 dbu = 96 + 288).

Table 1: Inferred vs. ground-truth grid parameters (all match exactly).

Design	Rails	Straps (width / pitch / offset [/ spacing])	connect
sky130hd/gcd	met1 0.480	met4 1.600/27.140/13.570; met5 1.600/27.200/13.600	{{met1 met4}} {{met4 met5}}
nangate45/gcd	metal1 0.170	metal4 0.480/56.000/2.000; metal7 1.400/40.000/2.000	{{metal1 metal4}} {{metal4 metal7}}
sky130hd/aes	met1 0.480	(same as sky130hd/gcd)	{{met1 met4}} {{met4 met5}}
asap7/gcd	M1, M2 0.018	M5 0.120/11.880/0.300/sp. 0.072; M6 0.288/12.000/0.513/sp. 0.096	{{M1 M5}} {{M5 M6}}

Note: asap7’s M1 and M2 are *both* (FOLLOWPIN) layers; M2 is an intermediate layer of the M1→M5 via stack, not a connect endpoint.

6.2 Round-trip verification: 4/4 exact

We re-ran the ORFS PDN step with only the inferred configuration (isolated output directory; inputs symlinked), re-extracted geometry, and compared wire rectangles and via sets. Wires are compared as box multisets with 1 dbu tolerance—this absorbs float-to-dbu rounding inside `pdngen`’s own arithmetic, not inference error—and vias as (net, layer-span, via-name, box) tuples. This tolerance applies only to the geometry-regeneration check; parameter recovery is judged by a separate, stricter test—every inferred parameter must equal the archived truth parameter in dbu integers. The geometric tolerance therefore cannot mask a parameter-level error: a wrong parameter either shifts geometry by a full dbu, which the 0-dbu-strict candidate validation catches, or leaves geometry unchanged, which is a genuine dual catalogued explicitly in §7. All four designs reproduce the original geometry *exactly*:

Design	Wires (orig / regen)	Vias (orig / regen)	Result
sky130hd/gcd	133 / 133	2,907 / 2,907	PASS
nangate45/gcd	65 / 65	279 / 279	PASS
sky130hd/aes	318 / 318	16,403 / 16,403	PASS
asap7/gcd	114 / 114	432 / 432	PASS

Because the inferred configurations omit `halo`, `rails_start_with`, and rail pitch/offset, the round trip also *empirically* proves these parameters have no geometric effect.

6.3 Variant experiments (sensitivity and dead parameters)

On sky130hd/gcd we additionally verified: (i) changing met4 pitch 27.14 → 30.00 changes geometry (2,907 → 2,601 vias) and re-inference recovers 30.000 exactly; (ii) flipping `stripes_start_with` to GROUND swaps the VDD/VSS grids and is recovered as GROUND; (iii) setting `rails_start_with` to GROUND, `halo` to 5, and rail pitch/offset leaves geometry bit-identical (133 wires / 2,907 vias), direct evidence of dead parameters.

6.4 Partial-region extrapolation

To test the partial-observation motivation, we cropped the sky130hd/gcd extracted geometry—the labeled ODB view, not imaged polygons—to three windows and re-ran inference on each crop alone. A 20%-width full-height vertical strip ($\sim 20\%$ die area) recovers every parameter exactly—met4 1.600/27.140/13.570, met5 1.600/27.200/13.600, `connect` and `stripes_start_with` all exact—and regenerates the full die: PASS. A $\sim 10\%$ -area square in the lower-left corner (3 stripes per layer per net in-window) likewise recovers everything exactly: PASS. But a thin 10%-width vertical strip fails: met5 is recovered exactly, yet the window holds only one VDD and zero VSS met4 stripes, so met4 pitch is unidentifiable and full-die regeneration FAILS—an information deficit, not a numerical error. Local recovery therefore depends on crop shape and on each (layer, net) presenting enough period instances (here, ≥ 2 –3 stripes sufficed). These are labeled-database crops; the harder label-free imaged-polygon setting is evaluated next.

6.5 Label-free attack: from raw polygons to spec

The results above assume the labeled ODB view (net identity, shape kinds, via spans). We now drop all labels: the input is per-layer polygons only, simulating a de-labeled layout database. A six-stage pipeline restores the labels before the unchanged inference runs: (1) connectivity tracing—shapes as nodes, geometric touch/overlap as edges (same-layer contact, via-to-metal overlap, stacked-via XY overlap), union-find yielding exactly 2 connected components (1,597 / 1,443 shapes), each of purity 2/2, in ~ 2 s; (2) PG-net identification—ranking components by wire count and bbox area (the two PG nets dominate any signal net; a synthetic test injecting 300 random signal wires still ranks the two PG nets top-2); (3) polarity identification—row orientation plus tap-cell library knowledge (N rows: VDD rail on top; public PDK information), voting 96/96 rails consistent on sky130hd/gcd and spot-checked on the other three designs (58/58, 228/228, 106/106 against truth); (4) rail/stripe classification—thin horizontal wires on the lowest metal layer are rails, the rest stripes (0 misclassifications vs. truth); (5) via-span recovery—cut-layer-to-metal mapping from the process stack plus overlap checks; (6) the unmodified `pg_infer.py` on the relabeled geometry.

Every parameter is recovered exactly, including `stripes_start_with=POWER`; the inferred configuration is *byte-identical* to the labeled pipeline’s output, and the round-trip regenerates 133/133 wires and 2,907/2,907 vias. Two qualifications apply. First, a fundamental one: a global VDD/VSS swap combined with a `stripes_start_with` flip is geometrically indistinguishable—a $\mathbb{Z}_2$ *naming* symmetry, broken here by PDK-anchored rail polarity; an attacker without PDK library knowledge recovers the spec only up to this $\mathbb{Z}_2$ equivalence class. Second, the input is a de-labeled ODB, not true imaged polygons: per-layer polygon attribution, cut-layer grouping, and stack order are assumed known, and layering noise (misalignment, missed/false cuts) is unmodeled—true imaging remains future work. (The DEF SPECIALNETS net names used for the final power/ground name strings are label leakage; geometry recovery does not depend on them.)

Foundry scenario: real GDSII input. The threat model names an untrusted foundry, which delivers GDSII—not a de-labeled ODB. We ran the same six-stage pipeline on a real GDSII stream-out of sky130hd/gcd (3,040 polygons on (layer, datatype) pairs; 6 TEXT net-name records of the kind klayout `produce-net-names` emits were detected and stripped before extraction, so the attacker input is pure polygons). Every verdict matches the ODB de-labeled run item-for-item: exactly 2 connectivity components (1,597/1,443), 96/96 polarity votes, 133/133 rail/stripe classification with 0 misclassifications, 2,907/2,907 via-name recovery from the `pdngen {cut}_{w}x{h}` naming rule, a byte-identical inferred configuration, and a 133/133, 2,907/2,907 round-trip. GDS

polygon order is by (layer, datatype), not by net—this exposed and fixed a real ordering-assumption bug in the union-find candidate enumeration (a $j > i$ dedup that systematically missed adjacent cut-layer via pairing under layer-grouped order), which the net-interleaved ODB input had masked. GDSII natively provides per-layer polygon attribution, which is exactly what the label-free pipeline assumes known; the unclosed gap is therefore only the *silicon* imaging gap (delaying noise: misalignment, missed/false cuts)—a separate scenario from the foundry GDSII one.

Name-strip: cell names are not a leak. A follow-up closes the cell-name question constructively. A pure-stdlib GDSII reader (no third-party parser) discards every name record—`STRNAME`, `SNAME`, `STRING`, `LIBNAME`, `PROPVALUE`—with an audit trail, and the attacker input is asserted integer-only: (layer, datatype, x1, y1, x2, y2) per rectangle. Adversarially renaming the structure to a via-cell-shaped leak name leaves all 3,040 extracted rectangles byte-identical, so name-independence holds by construction, not by accident. Via names are recovered purely geometrically: the (layer, datatype) pair maps to the cut layer via the attacker’s own process-stack table, and $w \times h$ is measured from the cut rectangles—2907/2907, with a byte-identical inferred configuration and a 133/133, 2,907/2,907 round-trip. All 16 pipeline verdicts match the earlier GDSII run item-for-item. Hierarchical GDSII (cell references, `SREF`/`AREF`) is not covered: the reader fails closed on references rather than expanding them.

6.6 Core rings and macro grids

Core rings and macro grids are the most complex, IP-valuable part of a PDN spec. We extend the methodology to both (labeled-database view, as in §§3–5).

Core ring. On sky130hd/gcd with a chameleon-style `core_ring` truth configuration (met4/met5: width 2.0, spacing 2.0, `core_offset` 4.0), ring bars are the unique wire rectangles whose centers fall outside the core’s short-axis range (stdcell straps extend to the ring’s outer edge but stay core-centered). All six parameters recover exactly (4-side spread 0 dbu), and the round-trip regenerates 141/141 wires and 2,989/2,989 vias. Inner ring is VDD, outer VSS (pg_nets order). `pad_offset` rings were not covered (no pads in this design).

Macro grid. On nangate45/gcd with a fakeram45_32x64 macro instance, macro straps recover 6/6 exactly (metal5/metal6: width 0.930, pitch 10.0, offset 2.000 from the macro edge, VSS/VDD order), and the connect chain `{metal4_PIN_ver metal5} {metal5 metal6} {metal6 metal7}` is fully recovered from via evidence (80 pin-to-strap vias identified by their $0.28\mu\text{m}$ pin-width via signature). Round-trip: 84/84 wires, 350/350 vias. Two qualifications: `blockages` is only observable up to superset equivalence (truth `metal1--4` vs. observed `metal4`—the round-trip with the reduced set is geometrically identical, confirming the ambiguity has no geometric consequence), and pin names plus the `orient` selector need LEF/DEF, not PDN geometry. Rotated macros and the `_PIN_hor` path are untested.

7 Identifiability Analysis

We began this study expecting the parameter→geometry mapping to be significantly non-injective, hoping to characterize large equivalence classes. The experiments say otherwise: the mapping is injective wherever the geometry constrains the parameters, and every observed failure of injectivity

has an analytic condition. We state this as a theorem; the randomized sweeps that follow test the implementation against it.

Theorem (identifiability of regular PDN grids). Let G be the map from `pdngen` configurations to placed grid geometry, and let $\sim$ identify configurations that differ only in dead parameters: `rails_start_with` (dead by exhaustive source inspection—a full-text search over the entire OpenROAD source tree at `f12e2f47` finds zero occurrences, and across all OpenROAD-flow-scripts the only hits are inert `set ::rails_start_with` lines in seven `pdn.cfg` user configs (settings nothing reads); configuration keys are accessed only as literals and `dict keys` enumerates layer names, excluding dynamic-key indirection; the parameter no longer exists in the current C++ `src/pdn/`), `halo` (macro-only), rail `pitch/offset` (dead by code inspection and case verification: re-adding them leaves geometry bit-identical), and the arbitrary grid name. On the quotient space of live parameters, G is injective—every live parameter is exactly recoverable from the geometry—under the following sufficient conditions:

- (C1) every observed (layer, net) pair presents at least two stripes, so pitch and offset are constrained;
- (C2) the leading offset satisfies $o \geq \text{width}/2$ on every strap layer, so no leading stripe is dropped;
- (C3) no explicit `spacing` S satisfies $S + W = \text{pitch}/2$, so the VSS phase cannot coincide with the default;
- (C4) every `connect` pair yields at least one via, so each pair leaves geometric evidence;
- (C5) parameters are quoted at dbu resolution, since distinct μm values rounding to the same dbu are indistinguishable;
- (C6) via stacks span their full declared ranges—connect identifiability requires chain integrity (deleting a whole span corrupts `connect`, cf. E3).

The geometry is always taken modulo two intrinsic symmetries: the $\mathbb{Z}_2$ naming symmetry (a global VDD/VSS swap plus a `stripes_start_with` flip is geometrically invisible, broken in practice by PDK-anchored rail polarity) and blockage-superset equivalence (`blockages` is observable only up to superset: `metal1--4` vs. `metal4` are geometrically identical). Outside (C1)–(C6), G fails injectivity in exactly the following catalog, each case analytic with an observed instance:

1. *Dropout-induced naming duality.* When the leading stripe is dropped (§3), the base-net naming can become unidentifiable, with an analytic existence condition (violates C2). Let the VSS phase shift be shift ($= \text{pitch}/2$ or $\text{spacing} + \text{width}$), and recall each net’s stripes are placed by its own loop with its own dropout trim. Then (POWER base, offset o) and (GROUND base, offset $o + \text{shift}$) generate dbu-identical labeled geometry exactly when $o \in [\text{width}/2 - \text{shift}, \text{width}/2]$: the POWER hypothesis drops its leading VDD stripe ($o < \text{width}/2$) while its VSS loop (offset $o + \text{shift} \geq \text{width}/2$) drops nothing, so its visible VDD set $\{o + k \cdot \text{pitch} : k \geq 1\}$ and visible VSS set $\{o + \text{shift} + k \cdot \text{pitch} : k \geq 0\}$ coincide stripe-for-stripe with the GROUND hypothesis’s undropped VSS and VDD sets (the far loop bound then applies identically to equal visible sets). Observed in the sweep and flagged `AMBIGUOUS`, e.g. truth (GROUND, $o=4966$) vs. dual (POWER, $o=846 = 4966 - 4120$) with $\text{width}/2=862$, $\text{shift}=4120$.
2. *The $(\neg \text{start}, o + \text{shift})$ duality fails outside the dropout window.* For $o \geq \text{width}/2$ the hypothesized dual (GROUND, offset $o + \text{shift}$) necessarily loses the boundary stripe that (POWER, o)

places at $ref + o$ (typical difference: 2 wires + 169 vias), by construction of the loop bound—so `starts_with` and offset are jointly identifiable there. The dropout window of case 1 is the exact exception. Confirmed by generating the literal dual of 553 exactly recovered configurations (0 identical) and 0/800 on the fresh-seed sweep; the argument itself is analytic, not statistical.

3. *Phasing coincidence.* When an explicit `spacing` S and width W satisfy $S + W = pitch/2$, the resulting VSS phase coincides with the no-`spacing` default, so the explicit-spacing and no-spacing configurations are geometrically identical (violates C3; confirmed 20/20 round-trip PASS).
4. *Single-stripe layers.* A layer carrying exactly one stripe per net leaves `pitch` unidentifiable: any pitch above a geometry-implied lower bound reproduces the same shapes (violates C1).
5. *Zero-yield connect pairs.* A `connect` pair that produces no vias is geometrically equivalent to omitting that pair entirely (violates C4). The mechanism is geometric overlap, not layer adjacency (§3): the spanning pair `{met1 met5}` yields *zero* vias because the parallel narrow `met1` rail cannot cover the wide `met5` stripe—`pdngen` skips every insertion (PDN-0042)—while the split pair `{met1 met4} {met4 met5}` yields the full 2,907-via stack, so spanning-vs-split is identifiable (connect-pair granularity is recoverable), but spanning-vs-omitted is a genuine equivalence. Note this zero-yield behavior was observed on the 2022 legacy Tcl implementation; the current C++ version turns it into a hard error (PDN-0179, see §8.5).
6. *Float-truncation near-duals.* `pdngen` serializes parameters as μm text (`%.3f`) and converts back via `microns_to_dbu` (round-half-up, confirmed by source inspection and probes); the forward map is deterministic in the dbu integer domain, and the two conversions crack at $\pm 2\text{dbu}$. A parameter within $\pm 2\text{dbu}$ of truth can therefore present identical observed geometry. These near-duals are resolved by re-validating every candidate hypothesis through the true generator with a 0-dbu strict wire/via diff: across 1,600 sweep configurations every selected candidate regenerates the observation bit-exactly, leaving zero failures (§7).
7. *dbu rounding classes.* Stripe geometry is computed in database units, so distinct μm values that round to the same dbu are indistinguishable (e.g. $30/30 \pm 0.35\text{dbu}$ variants geometrically identical). Recovery is therefore exact only down to the layout database’s unit grid, and parameter values must be quoted with that quantization in mind (C5).

Macro-grid parameters, pin names, and blockages are unrecoverable only when absent from the design (no macros, hence no macro-grid geometry to observe). The theorem’s conditions are sufficient and each exception is observed; “empirically identifiable” hedges one residual: untested joint transforms could in principle add equivalence cases, and the evidence scale is stated below.

Randomized sweeps: implementation correctness against the forward model. The theorem above is analytic; the sweeps do not establish it but certify that the implementation realizes it. A sweep is a differential test of the inferer against the true generator, and its value is measured in the bugs it caught: the default-vs-explicit `spacing` decision must use *exact* integer comparison (`shift == pitch/2`)—a tolerance misclassified a 1 dbu deviation and shifted a whole net, since 1 dbu is not noise on the parameter side; the union-find candidate enumeration assumed net-interleaved input order and systematically missed adjacent cut-layer via pairing under layer-grouped GDSII order—exposed only by the real GDSII run (§6.5); and the C++ port’s pilot runs hit the new generator’s validity domain (a 5 dbu manufacturing grid, $pitch \geq 2(width + spacing)$ with explicit

spacing, and via-enclosure width floors), all hard errors in the current build (see §8.5). With the implementation fixed, the sweeps’ final numbers are the certificate.

To move beyond four hand-picked designs, we swept `pdngen` over random configurations on sky130hd/gcd, sampling width, pitch, offset, spacing, and `stripes_start_with` over legal ranges in real `pdngen` runs. A first pass (91.5% dbu-exact) exposed one systematic cause for all 66 inference failures—`pdngen` drops any stripe not fully inside the core (§3), flipping the lowest visible stripe’s net and tripping the cross-layer `starts_with` consistency assertion. We closed it: the inference now runs a dropout-aware correction loop—naive least-squares fit, then forward validation with a dbu-exact forward model *including* the dropout rule, then hypothesis search over (base net, number k of dropped leading stripes), keeping every globally cross-layer-consistent hypothesis and flagging `AMBIGUOUS` when more than one survives. Because that rule was tuned on those very failures, we re-ran the sweep on a *fresh seed* (20260929) as an out-of-sample test: all 800 ground-truth configurations are archived in the public repo under `reports/rev5-scan-truth/`, and every configuration’s round-trip was actually re-run—no determinism argument. We report three strictly defined metrics:

Metric	Definition	Result ($N = 800$)
<i>inference success</i>	inferred emits a configuration	800/800 (100%)
<i>round-trip pass</i>	re-running <code>pdngen</code> , wire/via sets identical (all measured)	800/800 (100%)
<i>exact</i>	every parameter dbu-identical, <code>starts_with</code> consistent, and round-trip passes	798/800 (99.75%)
<i>equiv</i> (genuine geometric dual)	≥ 2 parameterizations, identical geometry, round-trip passes	2/800
<i>failure</i>	no configuration emitted, or round-trip fails	0

The two non-exact cases: `cfg_0462`, flagged `AMBIGUOUS`—a dropout-induced naming dual (truth (GROUND, $o=4966$) vs. dual (POWER, $o=846=4966 - 4120$); analytic condition in case 1 above); and `cfg_0578`, a generalized reparameterization (`starts_with` flip plus offset/shift re-expression, geometrically identical, round-trip bit-exact). The dropout-correction branch triggered on 60/800 configurations (7.5%)—close to but not identical with the original seed’s 66/800 (8.25%): the trigger rate is seed-dependent, as expected, since dropout fires exactly when the sampled offset falls below $width/2$ and different seeds sample different offset fractions. Width and pitch recover with zero error on all 800. (The dropout threshold is $offset < width/2$ on both strap layers, confirmed by targeted Tcl bisection; an independent pre-fix scan with $N = 630$ gave consistent 92.2%.)

Breadth and float-truncation modeling: 800 more configurations, failures to zero.

The sweep above varies only stripe parameters on one PDK. We extended it to nangate45/gcd and asap7/gcd ($N = 300$ each; stripe parameters plus `connect` pairs, rail widths, and strap-layer selection) and to sky130hd/aes ($N = 200$; a larger floorplan, $\sim 11k$ – $25k$ shapes per configuration versus $\sim 1k$ for gcd), with all truth configurations archived (`reports/rev5-breadth-truth/`). The breadth sweep initially left 3 failures, all from one generator-side mechanism: `pdngen` evaluates stripe positions in floating-point micron and truncates to dbu, so an integer-dbu truth parameter can produce observed geometry 1–2 dbu off; the inferrer (integer-dbu arithmetic) then locks onto the off-by-one parameterization, which the strict round-trip catches. We closed this with an explicit truncation model: after the initial inference, hypotheses are jittered by ± 1 – 2 dbu per parameter on the serialization grid (hypotheses only—the archived truth is never consulted), and every candidate

is re-validated through the true generator with a 0-dbu strict wire/via diff; the observation-faithful candidate is selected. All 3 failures resolve to bit-exact (nangate45 cfg0046, asap7 cfg0004 and cfg0160); the 5 genuine geometric duals are retained (sky130hd/gcd cfg0462, cfg0578; asap7 cfg0161; sky130hd/aes cfg0077, cfg0186). Combined over both sweeps (1,600 configurations): inference success 1600/1600, round-trip pass 1600/1600, exact 1595/1600 (99.69%), 5 genuine duals (0.3%), *zero failures*. Not covered: full-synthesis larger designs (ibex/jpeg/ariane), single- or triple-strap-layer stacks, and directed high-pitch probes.

Equivalence-class catalog. Directed probes confirm the theorem’s degenerate cases: **P** (explicit spacing with $S + W = pitch/2$ vs. omitting it—20/20 round-trip PASS); **S** (μm values rounding to the same dbu collide—30/30 ± 0.35 dbu variants geometrically identical); the **single-stripe** boundary (pitch genuinely unidentifiable with one stripe per net); and the **dropout-induced naming duality** (analytic condition in case 1 above; observed cases flagged **AMBIGUOUS**). Across both sweeps (1,600 configurations), genuine geometric duals total 5 (0.3%), including generalized reparameterizations (**starts_with** flip plus offset/shift re-expression). A hypothesized offset-translation class (sw, o) vs. ($sw, o + p$) is falsified (finite core loses exactly one boundary stripe).

8 Defenses: Anti-RE PDN Obfuscation

The attack succeeds because rule-generated PDNs are perfectly regular: uniform pitches, deterministic VSS phasing, and complete via stacks at every crossing make the forward model invertible. A natural counter is *anti-RE PDN obfuscation*: grids engineered so that the inference assumptions fail, while the PDN remains functional. Hardware obfuscation has a rich literature at the logic level (camouflaging, logic locking [8, 16]), in split manufacturing [7, 15], and in design-for-trust generally [14]; physical-design watermarking [17] and power-signature watermarking [18] hide information in layout and power networks respectively, and layout-level anti-RE techniques [11] are the closest defense-side analog to our decoy grids. On the attack side, our work is most directly preceded by ReGDS [10], which recovers gate-level netlists from GDSII at 100%—we invert the same “layout as evidence” threat model one level up, recovering generator *parameters* rather than netlists—and the power grid’s security relevance is independently established by power-grid-aware side-channel analysis [12] and PDN-impedance tamper sensing [13]. A systematic search found no peer-reviewed prior work on obfuscation of the on-chip PDN structure itself. (Search scope: IEEE Xplore and ACM Digital Library, 2000–2026, keywords “PDN/power grid obfuscation,” “power grid anti-reverse engineering,” “PDN camouflaging”; hardware-security venues HOST, CHES, DAC, ICCAD, DATE, TCAD, TVLSI.) (Prior metal-mesh work targets invasive probing and tamper evidence—conductive shield layers that detect decapsulation—a different threat model from rule extraction, which is not PDN-grid obfuscation prior art.) On the synthesis side, the EDA literature addresses the *forward* problem—sizing and optimizing power grids under IR-drop and electromigration constraints (textbook treatment in [3] and the DAC/TCAD PDN-synthesis literature); we found no work inverting a production generator’s rule parameters from placed layout geometry, which is the inverse problem studied here. On the inference side, prior RE work exploits layout regularity mostly to recover *instances*, not generator parameters: Degate recognizes standard cells by approximate template matching against a library of known layout patterns, and ML-based gate recognition classifies imaged structures—both stop at identifying what is on the silicon, without asking which *rules* produced it. The closest conceptual precedent is inverse procedural modeling in graphics: Stava et al. recover procedural tree-model parameters from polygonal trees via MCMC optimization [19], and Bokeloh, Wand, and Seidel link partial symmetry in 3D models to inverse

procedural modeling [20]. Those targets are stochastic or visually approximate models; a PDN generator is exact and deterministic, which is what makes parameter recovery here bit-exact rather than approximate. Our work inverts one level further: from placed geometry back to the generator’s rule parameters. On the industrial side, parametric PDN templates are standard practice, which is why the transfer question matters: Synopsys ICC2’s `create_pg_mesh_pattern` (per-layer width/spacing/pitch/offset/trim) and Cadence Innovus’s `addStripe` power-planning flow both express the grid as a reusable parameter template—precisely the class of object our geometry→bundle inference consumes (§8.5).

Defense goal and methodology. The defender’s goal is to make the grid *specification*—not the geometry, which an attacker can always clone—unrecoverable or ambiguous, while keeping the PDN functional: fully connected, DRC-clean, and with no significant IR-drop degradation. Naming: we label experiments E1–E3 and the underlying defense mechanisms D1–D4—D1: decoys without vias or net attachment, D2: pitch jitter, D3: interleaved decoy grids, D4: via dropout—so E1 evaluates D3, E2 evaluates D2, and E3 evaluates D4. We denote the concrete interleaved-decoy instance evaluated here (36 extra via’d stripes at $+P/4$) E1a. We evaluate candidate obfuscations on sky130hd/gcd against two attacker models: the naive deterministic inference of Section 4 (zero modifications), and adaptive attackers that know a defense may be present. Defender-side checks are: (i) connectivity of the wire/via graph per net (a single component, no dangling vias); (ii) LEF minimum spacing; (iii) a lumped resistive mesh (LEF `RPERSQ`, uniform 1 A rail-current injection, ideal met5 feed, 5Ω nominal per-via resistance—an assumed nominal value) used strictly as a *comparative* IR metric, never as signoff. Experiments manipulate the extracted geometry directly (the complete input of `pg_infer.py`), which is equivalent to ODB-level edits for every attacker-side measurement; the unmodeled gap is that signal-wire DRC around decoys is not modeled, though decoys sit in the $+P/4$ gaps ($5.19\mu\text{m}$) far from any DRC risk region, with LEF spacing rules verified.

8.1 E1: Interleaved decoy grids (effective)

Setup. On sky130hd/gcd (met4 pitch 27140dbu, met5 pitch 27200dbu), every true stripe is duplicated at $+P/4$ with the same width, the same net, and a full copy of its via stack (met1–met4 stack plus the met4–met5 crossing vias), faithfully simulating a second `pdngen` pass interleaved with the first; decoy×decoy same-net met4–met5 vias are added as well. Stripes obeying `pdngen`’s loop bound yield 36 extra decoy stripes (18 per layer): wires $133 \rightarrow 169$, vias $2,907 \rightarrow 5,994$.

Attacker side. The naive inference fails loudly: with the least-squares fit of §4, met4 VDD reports pitch 13570.0dbu with a maximum residual of 3571.1 dbu, far above the 2-dbu gate (`AssertionError`; no parameters emitted). The exact-recovery attack of Sections 4–6 is defeated outright—for the naive attacker.

Adaptive attacker: multi-sequence inference. An attacker who knows decoys may be present upgrades the pitch fit to k arithmetic sequences per (layer, net): P from the modal adjacent difference, phases from circular clustering of $(c \bmod P)$, each cluster validated as an arithmetic progression. With $k = 2$ this attacker recovers *all 73 stripe centers exactly*—met4 $P = 27.140\mu\text{m}$ with phases at truth and truth $+P/4$ (19/19 VDD, 18/18 VSS centers), met5 $P = 27.200\mu\text{m}$ likewise (18/18 per net). The prediction is confirmed: both attack goals of Section 1 (extrapolation,

regeneration under modification) need only this parameter bundle, not knowledge of which sequence is original. D3 therefore does *not* prevent geometry cloning.¹

What D3 does protect: intent attribution. Even with the full two-sequence bundle recovered, the attacker cannot determine which sequence is the designer’s original grid. We stress-tested this against an attacker armed with the forward model itself: each of the 6 enumerated stripe subsets (met4 VDD: two true 9-stripe truncations plus the 9-stripe decoy grid, and the full 10-stripe true grid; met4 VSS: the 9-stripe true grid and the 9-stripe decoy grid) was forward-fit for exact reproduction by some `pdngen` parameterization, using a per-net loop semantics verified to rebuild both true grids stripe-for-stripe. Five of six are *legal* complete outputs—both true grids, both decoy grids, and the VDD drop-first truncation (indistinguishable from a complete 9-stripe grid at offset $+P$); only the VDD drop-last truncation is illegal (that offset would also emit the 10th stripe). Pairing candidates across nets under `starts_with=POWER` with the $VSS = VDD + P/2$ phase and the loop bound leaves *exactly two* consistent design-level hypotheses: the all-true design and the all-decoy design. Via evidence does not break the tie either: across the four (true/decoy met4) $\times$ (true/decoy met5) same-net crossing classes, via presence is 100% in all four (pooled 171/171, 171/171, 162/162, 162/162; maximum difference 0)—the decoy construction replicates via stacks faithfully enough that via patterns leak nothing about which grid is original (this symmetry was measured on our carrier, where the baseline places vias at every same-net crossing; it is a measured property of this carrier, not a theorem). A defender could in principle invert the majority (more decoy than true stripes) so that “largest AP is the original” heuristics fail—we did not implement or test this; it remains an untested hypothesis. This is **genuine intent ambiguity**: five subset candidates each admit a valid `pdngen` parameterization, collapsing at design level to exactly two consistent hypotheses—the all-true and the all-decoy design—and the attacker cannot determine which was the designer’s original. (This is a different object from the configuration-level equivalence classes hunted in Section 7—there, two generator configurations mapping to the same geometry; here, two subsets of one defended geometry each admitting a valid generator configuration.) In sum: D3 blinds the naive attacker and reduces the adaptive attacker from exact *attribution* to ambiguity; it does not impede geometry recovery at all. The value of intent attribution is narrow: it matters only when the attacker’s objective requires the designer’s *original* intent rather than any functionally equivalent grid—a condition we do not rely on for the main results; for the two primary attack goals of Section 1—extrapolation and regeneration under modification—it offers nothing.

Defender side. VDD and VSS each remain a single connected component with zero dangling vias. Minimum stripe spacing is $5.185\,\mu\text{m}$ on met4 ($\geq 0.3\,\mu\text{m}$ LEF) and $5.2\,\mu\text{m}$ on met5 ($\geq 1.6\,\mu\text{m}$ LEF). The mesh reports worst-case drop $66.90 \rightarrow 25.98\,\text{mV/A}$ and mean $38.89 \rightarrow 11.63\,\text{mV/A}$ —but this -61.2% is *fully explained by doubled metal*, not by obfuscation. Against an equal-metal unobfuscated baseline (met4/met5 pitch halved to $P/2$, no decoys; 169 wires, 5,994 vias, stripe length $18,918.4\,\mu\text{m}$ —identical to E1a):

¹Strictly, this parameter bundle is not a single legacy-`pdngen` configuration: feeding two `specify_grid stdcell` blocks silently ignores the second (one `PDN-0013` in the log; first-writer-wins), and running `pdngen` twice rips up the first grid. E1a’s interleaved geometry was constructed geometrically, which is all the attacker needs—the bundle regenerates shapes directly.

	Baseline	E1a (D3 decoys)	$P/2$ equal-metal
wires	133	169	169
vias	2,907	5,994	5,994
stripe length (μm)	9,588.6	18,918.4	18,918.4
worst IR (mV/A)	66.90	25.98	24.51
mean IR (mV/A)	38.89	11.63	11.54

Note: all IR figures in this table are qualitative—uniform-current, ideal-feed mesh model; comparative only, not signoff.

The decoy grid shows no IR advantage over the equal-metal baseline in this coarse model (worst 25.98 vs. 24.51 mV/A, mean 11.63 vs. 11.54 mV/A—differences this small carry no quantitative weight under uniform-current, ideal-feed modeling, so the comparison supports only “no benefit,” not a precise percentage). There is therefore no “negative physical cost”: D3’s real price is $2\times$ routing blockage on met4/met5 and $2.06\times$ via count, for no IR benefit over simply halving the pitch.

8.2 E2: Pitch jitter (coarsens recovery, does not blind)

Setup. Each met4 stripe and its full via stack are translated by $\delta \sim \text{Uniform}(-j, +j)$ for $j \in \{0, 2\%, 5\%, 10\%, 15\%\}$ of pitch, three seeds each (15 attempts). Stripes violating LEF spacing or via landing (vias must stay within the rail/met5 perpendicular overlap extent) are resampled—2 runs at $j = 15\%$ —so the 13 final runs are all physically legal.

Attacker side. The naive inference breaks at $j \geq 2\%$ (its ± 2 dbu arithmetic-sequence assertion is far exceeded). But a ~ 30 -line robust least-squares pitch fit—the obvious attacker upgrade—still recovers pitch with mean relative error 0%, 0.14%, 0.42%, 0.40%, 0.50% (max 0%, 0.21%, 0.76%, 0.97%, 1.19%) at $j = 0\%, 2\%, 5\%, 10\%, 15\%$ ($j = 0\%$ is the unmodified pipeline self-check), and offset within $2.97 \mu\text{m}$ at 15%. Jitter’s ceiling is therefore coarse, not exact, recovery: pitch within 1.2% but offset within $2.97 \mu\text{m}$ —about 11% of pitch, a substantial absolute error—which may suffice for a foundry cloning the grid but not for hiding the design intent.

Defender side. All 13 runs keep a single connected component per net, zero dangling vias, minimum spacing $\geq 5.68 \mu\text{m}$, and mesh worst-case drop essentially unchanged from baseline (within the coarse model’s noise; we claim only “electrically free” qualitatively, not a precise percentage)—jitter is electrically free, but buys little security against an adaptive attacker.

8.3 E3: Via dropout (falsified as a defense)

Deleting $p \in \{10\%, 30\%, 50\%\}$ of the 912 rail→strap via stacks (2,736 vias) leaves **connect** inference *unchanged* ($\{\{\text{met1 met4}\} \{\text{met4 met5}\}\}$): recovery operates on via *span types*, not instances, so instance deletion cannot move the attacker’s output—while mesh worst-case drop degrades by up to +426%. Deleting the entire met2–met3 span does corrupt **connect** (mis-inferred as $\{\{\text{met4 met5}\}\}$), but it fragments the PDN into 49 components per net and disconnects all 192 rail injection nodes; an analytic projection shows the mis-inferred configuration would regenerate with zero rail→strap stacks. Via dropout is self-defeating: a falsified defense, kept only because it quantifies that connect identifiability depends on via-chain integrity (Section 7).

8.4 Negative controls

We report three weak mechanisms rather than hiding them. **Floating decoys** (stripes with no net) never enter inference at all: our extractor iterates only POWER/GROUND nets, so they are filtered before inference begins. **Via-less decoys** (same net, no vias) crash the naive inference, but a one-line adaptive filter—dropping stripes with no vias—restores *exact* recovery of every parameter, so this defense is falsified. **Half-pitch** ($+P/2$) **decoys** are physically unrealizable under the no-spacing phasing used here: the VSS grid sits exactly at $+P/2$ from VDD, so such decoys coincide with the opposite net’s stripe centers—a hard short—and were never run.

8.5 Summary and limitations

Mechanism	Naive attacker	Adaptive attacker	Physical cost
D3 interleaved decoys (E1)	crashes (resid. 3571 dbu)	geometry exact via 2-seq. fit; 2-way <i>intent</i> ambiguity (all-true vs all-decoy)	no IR benefit vs. equal-metal; $2\times$ blockage, $2.06\times$ vias
D2 jitter (E2)	breaks at $\geq 2\%$	pitch err. $\leq 1.2\%$; offset err. $\leq 2.97\mu\text{m}$ ($\sim 11\%$ of pitch)	IR qualitatively unchanged
D4 via dropout (E3)	connect unchanged	wire params unaffected	IR $+42\%$ – $+426\%$; self-defeating
D1 no-via decoys	crashes	exact recovery (falsified)	none
D1 floating decoys	no effect	—	wasted routing

Note: IR and physical-cost figures above are qualitative (uniform-current, ideal-feed mesh; geometry post-processing, no routing/timing measurement)—comparative only.

Limitations. The resistive mesh is a comparative first-order model (uniform current, ideal met5 feed, nominal 5Ω vias, no packaging or EM analysis)—it cannot be presented as signoff, and no commercial IR-drop/EM signoff is available in this environment. The routability cost of D3 is now quantified against an equal-metal baseline: $2\times$ routing blockage on met4/met5 and $2.06\times$ via count, with no IR benefit over the unobfuscated $P/2$ grid in our coarse model; a full-flow routing congestion/timing measurement remains future work. The main attack experiments (§4–§6) use a toy design (gcd) and OpenROAD f12e2f47 (2022) with the legacy Tcl pdngen (pdngen::specify_grid). Only three distinct hand-picked grid configurations are exercised (sky130hd/gcd and aes share one), but the randomized sweeps (§7) cover 1,600 configurations across three PDKs and two design sizes, and core rings and macro grids are covered in §6.6; pad_offset rings and rotated macros remain future work. Commercial generators: we have no license for tool measurement, so we analyzed transferability at the parameter-semantics level instead. Innovus addStripe (documented in public tutorials and community sources) generates the same geometric object class our inference consumes—a periodic set of equal-width parallel stripes over a rectangular area, one stripe per listed net per set—and its options map onto our geometry-sufficient bundle almost 1:1: -width, -set_to_set_distance, -start_offset, -spacing, -nets, -direction, -layer and -area play the roles of width, pitch, offset, spacing, net polarity, direction, layer and generation domain respectively. The geometry→bundle stage of the attack is therefore tool-agnostic in principle; the bundle→config stage is a table lookup. The hard boundary is via/connect inference, which does *not* transfer: addStripe draws stripes only, while vias are generated by a separate sroute step with its own parameters, so our via-stack and connect recovery stays pdngen-specific. Datum conventions (edge vs. centerline), grid snapping, and boundary truncation are unmeasured. Transferability is analyzed at parameter-semantics level; tool measurement is future work. The defenses D2/D3 were

evaluated as geometry post-processing steps, not through a modified PDN flow—deploying them in practice requires flow integration (decoy insertion inside `pdngen` or as a post-PDN ECO step), which we did not implement. To check that the exact-recovery mechanism is not a legacy-only artifact, we rebuilt current OpenROAD from source (HEAD 80c6be9, September 2026) and re-ran the full round-trip on `sky130hd/gcd` under the new C++ API (`define_pdn_grid / add_pdn_stripe / add_pdn_connect`), using the 2022 ORFS sky130hd LEF as input data. The inferred configuration regenerates the truth geometry *exactly*: 3,051/3,051 shapes (134 wires + 2,917 vias), zero symmetric difference, with met4/met5 widths, pitches, and offsets recovered to the unit grid (met4: 1.600/27.140/13.570; met5: 1.600/27.200/13.600). To preempt confusion: both versions’ ground-truth configurations use the *same* offset parameter values (met5 offset 13.6), and the recovered 13.600 in each case is relative to that version’s reference frame—legacy (10120, 10640) dbu from Eq. (1) vs. the current version’s core-area corner (10120, 10880) dbu, a 240 dbu shift in y . The inference forward model is adapted per version, so “exact recovery” means exact recovery of the parameter *values*; the 133→134 wire difference is the geometric consequence of the same values under the shifted datum (one stripe moves in or out of the loop bound). The 3,051/3,051 figure is a within-new-version round-trip: regeneration with the inferred configuration compared against the new version’s own extracted geometry. A successful `{met1 met4}` connect now yields a *complete* intermediate via stack (912 vias per met1–met2, met2–met3, and met3–met4 step, 181 for met4–met5). Generator quirks differ from the 2022 version in detail: the non-adjacent `{met1 met5}` connect is now a hard error ([ERROR PDN-0179] “unable to repair all channels”) rather than a silent zero-via skip; `-verbose` and `-pin_direction` are explicitly marked deprecated (parsed but with no geometric effect); `add_pdn_stripe -width 0` is still rejected (PDN-0106, minimum width $0.3000\,\mu\text{m}$). Beyond the single design, we re-ran the full randomized sweep on the current C++ build: 200 configurations on `sky130hd/gcd` (seed 20260930; sampling restricted to the new generator’s validity domain—a 5 dbu manufacturing grid, pitch $\geq 2(\text{width} + \text{spacing})$ with explicit spacing, and via-enclosure width floors of $1.18\,\mu\text{m}$ on met4 and $0.26\,\mu\text{m}$ on rails, all hard errors outside the domain) reach 200/200 inference, 200/200 measured round-trip, and 200/200 exact parameter recovery (100%). The E2 jitter defense was also reproduced on HEAD: naive inference crashes at $j \geq 2\%$, while the robust least-squares fit recovers pitch within 1.64% at 15% jitter, matching the legacy conclusion. E1 and E3 remain validated on the 2022 version only. In short: the legacy-Tcl results are archival (the Tcl implementation has been removed upstream), and exact recovery is re-confirmed on the current C++ generator. Finally, these defenses protect the grid *specification* (design IP) against rule extraction; they do not stop an attacker who simply clones the geometry—a different threat model. We also note the attack surface widens on commercial generators, which our test set does not cover by measurement.

9 Conclusion

We characterized the identifiability of a regular PDN generator’s rule parameters from layout geometry, using OpenROAD’s `pdngen` as the verified vehicle. The parameter-to-geometry map proved identifiable on the live-parameter quotient space, with every degenerate regime characterized analytically in §7: exact recovery on four designs across three PDKs with wire-for-wire, via-for-via round-trip regeneration (4/4), confirmed not to be a legacy-only artifact by a 200/200 exact sweep on current OpenROAD (HEAD 80c6be9); randomized sweeps of 1,600 configurations with archived truth reach 1595/1600 exact (99.69%) with zero failures—inference and round-trip 1600/1600—the remaining cases falling into the explicitly characterized degenerate regimes. We closed the label gap (de-labeled ODB polygons and real GDSII suffice, up to a $\mathbb{Z}_2$ naming symmetry; silicon imaging

noise remains future work), extended recovery to core rings and macro grids, and demonstrated end-to-end that the recovered spec regenerates a legal PDN on a modified floorplan where naive geometry copy fails—the spec is executable intent, the geometry a one-time instance. For defenders, the takeaway is negative: regular PDN grids leak their complete construction rules, and this leakage can be engineered into *intent* ambiguity at best—interleaved decoy grids leave an adaptive attacker unable to tell which of two equivalent grids is original, though this matters only when the attacker needs the designer’s original intent—but not into geometry unclonability, and not for free (no IR benefit over an equal-metal unobfuscated grid); cheap fixes (jitter, via dropout) do not suffice. For the open-source EDA community, the verified forward model (now including the dropout rule) and the dead-parameter findings (notably `rails_start_with`, dead by exhaustive source inspection) are documented for the record; the legacy Tcl implementation that carried them has since been removed, so the finding is archival rather than actionable.

References

- [1] T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Lee, U. Mallappa, M. Neseem, et al. Toward an open-source digital flow: First learnings from the openroad project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, pages 1–4, 2019.
- [2] T. Ajayi, D. Blaauw, T. B. Chan, C. K. Cheng, V. A. Chhabria, D. K. Choo, M. Coltella, S. Dobre, R. Dreslinski, M. Fogaça, et al. OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain. In *Proc. GOMACTECH*, pages 1105–1110, 2019.
- [3] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu. *VLSI Physical Design: From Graph Partitioning to Timing Closure*. Springer, 2011.
- [4] R. Torrance and D. James. The state-of-the-art in IC reverse engineering. In *Cryptographic Hardware and Embedded Systems—CHES 2009*, volume 5747 of *Lecture Notes in Computer Science*, pages 363–381. Springer, 2009.
- [5] M. Fyrbiak, S. Strauß, C. Kison, S. Wallat, M. Elson, N. Rummel, and C. Paar. Hardware reverse engineering: Overview and open challenges. In *2017 IEEE 2nd International Verification and Security Workshop (IVSW)*, pages 88–94. IEEE, 2017.
- [6] S. E. Quadir, J. Chen, D. Forte, N. Asadizanjani, S. Shahbazmohamadi, L. Wang, J. Chandy, and M. Tehranipoor. A survey on chip to system reverse engineering. *ACM Journal on Emerging Technologies in Computing Systems*, 13(1), Article 6, pages 6:1–6:34, 2016.
- [7] T. D. Perez and S. Pagliarini. A survey on split manufacturing: Attacks, defenses, and challenges. *IEEE Access*, volume 8, pages 184013–184035, 2020.
- [8] K. Shamsi, M. Li, K. Plaks, S. Fazzari, D. Z. Pan, and Y. Jin. IP protection and supply chain security through logic obfuscation: A systematic overview. *ACM Transactions on Design Automation of Electronic Systems*, 24(6):65:1–65:36, 2019.
- [9] U. J. Botero, R. Wilson, H. Lu, M. T. Rahman, M. A. Mallaiyan, F. Ganji, N. Asadizanjani, M. M. Tehranipoor, D. Woodard, and D. Forte. Hardware trust and assurance through reverse engineering: A tutorial and outlook from image analysis and machine learning perspectives. *ACM Journal on Emerging Technologies in Computing Systems*, 17(4):1–53, 2021.

- [10] R. S. Rajarathnam, Y. Lin, Y. Jin, and D. Z. Pan. ReGDS: A reverse engineering framework from GDSII to gate-level netlist. In *Proc. IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 154–163, 2020.
- [11] S. Chen, J. Chen, D. Forte, J. Di, M. Tehranipoor, and L. Wang. Chip-level anti-reverse engineering using transformable interconnects. In *Proc. IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFTS)*, pages 109–114, 2015.
- [12] X. Wang, W. Yueh, D. B. Roy, S. Narasimhan, Y. Zheng, S. Mukhopadhyay, D. Mukhopadhyay, and S. Bhunia. Role of power grid in side channel attack and power-grid-aware secure design. In *Proc. 50th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2013.
- [13] T. Mosavirik, S. K. Monfared, M. Saadat-Safa, and S. Tajik. Silicon echoes: Non-invasive Trojan and tamper detection using frequency-selective impedance analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):238–261, 2023.
- [14] J. Rajendran, V. Jyothi, O. Sinanoglu, and R. Karri. Regaining trust in VLSI design: Design-for-trust techniques. *Proceedings of the IEEE*, 102(8):1266–1282, 2014.
- [15] F. Imeson, A. Emtenan, S. Garg, and M. Tripunitara. Securing computer hardware using 3D integrated circuit (IC) technology and split manufacturing for obfuscation. In *Proc. 22nd USENIX Security Symposium*, pages 495–510, 2013.
- [16] J. Rajendran, Y. Pino, O. Sinanoglu, and R. Karri. Security analysis of logic obfuscation. In *Proc. 49th ACM/EDAC/IEEE Design Automation Conference (DAC)*, pages 83–89, 2012.
- [17] A. B. Kahng, J. Lach, W. H. Mangione-Smith, S. Mantik, I. L. Markov, M. Potkonjak, P. Tucker, H. Wang, and G. Wolfe. Constraint-based watermarking techniques for design IP protection. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 20(10):1236–1252, 2001.
- [18] D. Ziener and J. Teich. Power signature watermarking of IP cores for FPGAs. *Journal of Signal Processing Systems*, 51(1):123–136, 2008.
- [19] O. Stava, S. Pirk, J. Kratt, B. Chen, R. Měch, O. Deussen, and B. Benes. Inverse procedural modelling of trees. *Computer Graphics Forum*, 33(2):118–131, 2014. (Presented at Eurographics 2014.)
- [20] M. Bokeloh, M. Wand, and H.-P. Seidel. A connection between partial symmetry and inverse procedural modeling. *ACM Transactions on Graphics*, 29(4):104:1–104:10, 2010. (Proc. SIGGRAPH 2010.)